In this document we show exactly where in an **Automotive Ethernet switch pipeline level**:

- stream identification happens,
- VLAN/FDB lookup happens,
- PSFP/Qci policing happens,
- Qav/Qbv queueing happens,
- and where reservation state is actually consumed in hardware.

There is **no single universal vendor pipeline**, but the logical order in a VLAN-aware automotive Ethernet switch is the IEEE 802.1Q bridge model:

```
RX MAC / PHY
  ↓
Ingress Port actions
  ↓
Relay / Forwarding process
  ↓
Egress Port actions
  ↓
Queueing + Transmission Selection
  ↓
TX MAC / PHY
```

In that model, the **ingress port** does filtering, tag handling, and classification; the **relay** does forwarding/filtering; and the **egress port** does filtering/tag handling plus **metering, queuing, and transmission selection**. That is the backbone you should keep in your head while placing TSN features in the datapath.

1. A precise "where does each function live?" view

A practical TSN-capable automotive switch pipeline looks like this:

```
[1] RX parser / ingress classification
      ├─ parse DA/SA/VID/PCP/EtherType/IP/UDP...
      ├─ optional stream identification
      └─ ingress admit / basic drop / tag handling

[2] Relay / forwarding decision
      ├─ VLAN membership checks
      ├─ FDB lookup / multicast replication decision
      └─ choose egress port set

[3] Per-egress stream enforcement (before queue commit)
      ├─ PSFP/Qci stream filter
      ├─ max SDU check
      ├─ stream gate
      ├─ flow meter / policing
      └─ assign internal priority / traffic class

[4] Egress queueing
      ├─ put frame into selected egress queue
      ├─ class/stream shaping state applies
      └─ queue backlog builds here

[5] Transmission selection on egress
      ├─ Qav CBS eligibility
```

```
      ├─ Qbv gate open/closed
      ├─ strict priority / scheduler arbitration
      └─ optional preemption interaction

[6] TX MAC / PHY launch
```

The standards model does not say “all TSN logic is ingress” or “all TSN logic is egress.” Instead, it splits the dataplane into ingress, relay, and egress action sets, and then places stream filtering/metering/queueing/scheduling around that structure.

---

2. Where stream identification happens

Stream identification is the **earliest TSN-specific classification step**, because the hardware first has to decide, “what stream is this frame?” before it can apply per-stream gates, per-stream meters, or per-stream redundancy logic. IEEE material says the frame fields that can determine the `stream_handle` include ingress port, source/destination MAC address, VLAN ID, priority, IP addresses, protocol, and L4 ports. It also explicitly says the `stream_handle` is **not** used to choose the output port.

So in switch hardware, stream identification belongs in the **RX parse / classification block**, before forwarding treatment and before PSFP actions are selected. The switch is essentially doing:

```
header fields + ingress port  →  stream_handle / stream ID
```

That stream ID then feeds later hardware tables. This is why stream identification is best thought of as a **classifier** at the head of the QoS pipeline, not a forwarding lookup.

What that means in silicon

A real switch parser typically extracts:

- ingress port
- destination/source MAC
- VLAN ID / PCP
- EtherType
- optionally IP and UDP fields

Then a TCAM-like or exact-match rule says:

- “These fields match **stream CamFrontLeft**”
- “These fields match **stream BrakeCmd**”

From there, the hardware can fetch:

- stream filter instance
- gate ID
- meter ID

- counters
- internal priority/class mapping.

Important non-obvious point

Do **not** mentally place stream identification after forwarding. Forwarding decides **where** a frame goes; stream identification decides **how the bridge treats it**. Those are different pipelines, even if many chips implement them in the same front-end parser.

---

3. Where VLAN / FDB lookup happens

This is the **relay / forwarding process** in the middle of the bridge. IEEE's bridge model calls this **Relay (Action Set 2)** and describes it as **forwarding and filtering**. The MAC Relay Entity is the part that uses the bridge's forwarding information and filtering information to decide the egress port set.

So the order is:

1. ingress parser/classifier understands the frame,
2. relay logic checks VLAN and forwarding rules,
3. the bridge decides the egress port or replicated egress port set.

This is where:

- VLAN membership checks happen,
- filtering by VLAN/port state happens,
- FDB lookup happens,
- multicast replication decision happens.

It is **not** where Qav or Qbv lives. Those are egress scheduling functions, not relay lookup functions.

Silicon mental model

Think of the relay stage as asking:

"For this `{DA, VID}` on this ingress port, which egress ports are legal and should receive the frame?"

The answer is usually:

- one egress port for unicast,
- several for multicast,
- or none, if filtered.

Key separation to remember

A common mistake is to think stream ID drives the forwarding table. IEEE's stream-identification material explicitly says the `stream_handle` is not an output-port selector. So the actual datapath logic is:

```
stream ID/classification  → treatment selection
DA + VID + bridge state   → egress port set
```

Those are parallel but different decisions.

---

4. Where PSFP / Qci policing happens

This is the part many people place wrongly.

Per-Stream Filtering and Policing (PSFP, IEEE 802.1Qci) is logically **before the frame is committed to its egress queue**, because its whole job is to:

- drop malformed or oversized traffic,
- close off streams outside allowed windows,
- meter traffic against its stream profile,
- and optionally assign an internal priority value before queueing. IEEE material describes the stream filter with fields such as Stream ID, Priority, Filters, Meter ID, and Counters; the stream gate can be open or closed; and the flow meter uses bandwidth-profile parameters.

The stream filter/gate/meter chain is shown in the IEEE PSFP organization material as:

- stream identification,
- maximum SDU filtering,
- stream gate,
- flow meter,
- then queueing/shaping usage. It also notes that flow metering is applied **after** other filter specifications that could already discard the frame, and that `queueMaxSDU` is applied **after the stream filters**.

So the right mental placement is:

```
forwarding picked egress port
   ↓
per-stream filter on that bridge component / egress path
   ↓
gate / meter / IPV assignment
   ↓
queue insertion
```

Why this placement matters

If you put PSFP “after queueing” in your mental model, you have already let a misbehaving stream consume queue memory and potentially interfere with good traffic. PSFP exists to stop or shape the frame **before it becomes queue backlog**. That is exactly why it includes filters, gates, and meters tied to the stream.

What the hardware tables look like conceptually

For each stream filter instance, the hardware needs entries such as:

- Stream ID
- priority
- filter specs
- Gate ID
- Meter ID
- counters.

The PSFP material also states that stream gates can assign an **Internal Priority Value (IPV)** and that queueing handles the IPV assigned by stream gates. That means PSFP is not just “drop or pass”; it can also determine the internal traffic-class mapping used by the egress queueing machinery.

---

5. Where Qav / Qbv queueing and scheduling happen

These are **egress functions**, not ingress functions.

IEEE’s bridge dataplane summary puts **metering, queuing, and transmission selection** in the **Egress Port (Action Set 3)**. That is the correct anchor point for both Qav and Qbv.

5.1 Qav — Credit-Based Shaper (CBS)

Qav is a **per-traffic-class egress shaper**. IEEE overview material describes 802.1Qav as specifying the Credit-Based Shaper, and other IEEE material explains that CBS is configurable on a **per traffic class** basis and that the reserved bandwidth for an SR class is defined by `idleSlope`.

So Qav lives here:

```
selected egress queue
   ↓
credit accounting for that traffic class
   ↓
eligible / not eligible for transmission
```

It is not looking at the forwarding database. It is looking at:

- queue occupancy,
- configured class bandwidth,
- current credit state.

5.2 Qbv — Time-Aware Shaper (TAS)

Qbv is a **queue gate scheduler**. IEEE material says transmission from each queue is scheduled relative to a known timescale and that a transmission gate is associated with each queue; if the gate is open, queued frames can be selected for transmission, and if it is closed, they cannot.

So Qbv sits here:

```
egress queue
   ↓
time-aware transmission gate
   ↓
transmission selection / MAC
```

That is why Qbv is fundamentally different from PSFP:

- PSFP gates **streams**
- Qbv gates **queues / traffic classes**.

5.3 When Qav and Qbv are both present

IEEE material on egress modeling says CBS, TAS, and strict priority can be logically coupled **in series**, meaning a frame transmits only when all relevant elements allow it at the same time and a frame is actually available.

That means the effective egress decision is not “Qav or Qbv”; it is more like:

```
frame in queue
AND Qav says enough credit
AND Qbv gate is open
AND scheduler selects this queue now
→ transmit
```

That is the right hardware-level way to think about combined TSN shapers.

---

6. Where reservation state is actually consumed in hardware

This is the deepest part of your question.

The **reservation protocol itself** — SRP/MSRP in the distributed case, or CNC/CUC-style TSN configuration in the centralized case — is **not** the packet datapath. It is control/management logic that computes and installs datapath state. IEEE 802.1Qcc material says the CNC uses remote management to discover topology, retrieve bridge capabilities, and **configure TSN features in each bridge**; it can configure resource-allocation classes, traffic classes, and potentially transmission gate schemes, VLANs, active topologies, synchronization, and TSN domains.

So when you ask, "where is the reservation state consumed in hardware?", the answer is:

It is consumed **indirectly**, by the hardware tables and scheduler parameters that reservation/configuration installs.

The main places are:

6.1 Forwarding / topology state

Reservation or centralized TSN configuration may determine:

- VLANs,
- active topology,
- forwarding/path choices.
  That state is consumed in the relay/FDB/VLAN tables during the forwarding decision.

6.2 PSFP tables

Reservation/configuration installs:

- stream filter entries,
- stream gate entries,
- flow meter parameters,
- max SDU settings,
- internal-priority behavior.
  Those are consumed when the frame passes through the per-stream enforcement stage before queueing.

6.3 Qav queue parameters

For reserved stream classes, the hardware consumes per-class CBS parameters such as `idleSlope` and related shaper state. IEEE material ties reserved SR-class bandwidth directly to `idleSlope`, and the Credit-Based Shaper plus SRP is intended to provide bounded per-bridge latency.

6.4 Qbv gate control lists

If scheduled traffic is used, the hardware consumes the configured **transmission gate schemes** or gate control lists at the egress queue gates. IEEE Qcc overview material explicitly lists transmission gate schemes among the things a CNC may configure.

6.5 Traffic-class / queue mapping

Reservation/configuration also affects which internal traffic class a stream uses. That mapping is then consumed when the hardware chooses the target queue and transmission selection logic.

So the reservation state is not a mystical object “inside the packet.” It becomes concrete hardware state in:

- forwarding tables,
- stream filter/gate/meter tables,
- queue/shaper parameters,
- gate schedules,
- and internal class mappings.

---

7. A worked automotive example

Take a front camera stream entering on port 2.

Step A — ingress classification

The parser sees:

- ingress port 2
- source MAC = camera ECU
- VLAN 100
- PCP 5
- AVTP or UDP header pattern

It matches a stream-identification rule and assigns:

- `stream_handle = Cam_FL`
  This happens at the front-end classifier.

Step B — relay / forwarding

The relay engine looks up:

- destination MAC + VLAN 100

and determines:

- egress port 7 toward the central compute switch
  or several egress ports if this is multicast.
  This is FDB/VLAN processing, not PSFP.

Step C — per-stream enforcement

Before the frame is put into the egress queue for port 7, PSFP logic for `Cam_FL` checks:

- max SDU,
- stream gate state,
- flow meter state,
- possibly assigns internal priority/class.
  If the stream is outside its admitted envelope, the frame is dropped or marked here.

Step D — queueing

The frame is placed into the egress queue associated with its internal traffic class. This is where backlog for that class lives.

Step E — shaping and scheduled transmission

On port 7:

- Qav checks CBS credit for the reserved class,
- Qbv checks whether the queue gate is open in the current time slot,
- transmission selection chooses the queue if both conditions allow it.

Step F — actual reservation-state usage

What the packet is "using" from prior reservation/configuration is:

- the VLAN/FDB path state,
- the stream filter/gate/meter instance,
- the class mapping,
- the class shaper parameters,
- and the queue gate schedule.
  That is where reservation becomes real in silicon.

---

8. The clean mental split you should keep

Here is the shortest correct model:

```
Ingress:
  identify the stream

Relay:
  decide the egress port(s)

Pre-queue on egress:
  enforce per-stream rules (PSFP/Qci)
```

```
Queue:
  hold the admitted frame in the selected traffic class

Transmission selection:
  enforce per-class / per-queue rules (Qav/Qbv)

TX:
  launch
```

That is the right pipeline-level decomposition for a TSN-capable automotive Ethernet switch. It matches the IEEE 802.1Q bridge model and the TSN function split much better than the vague statement “TSN happens in the switch.”

---

9. One automotive caution

If you are working in an **AUTOSAR** context, remember that the full IEEE TSN toolbox is larger than what AUTOSAR currently exposes. AUTOSAR’s explanatory TSN document says AUTOSAR does **not** support time-aware shapers, active stream identification, or frame replication, while still assuming VLAN-aware switches. So the IEEE pipeline above is the full standards picture, but an AUTOSAR deployment may instantiate only a subset of it.